Maddy St. Charles
Climate 410, Lab 4
Due Monday, November 16th, 2026 at 5pm

Methodology

Task 1

The goal for Task 1 is ultimately to reproduce the graph given as Figure 1 in the lab manual. To do this, we begin with the simplest of models, only including the basic diffusion solver. This should be the blue parabola on the graph. To implement this, we were instructed to use Equation 2 from the lab manual, which is given below.

$$\overline{T}_{j+1} = L^{-1}\overline{T}_{j} \qquad \text{where } L = I - \lambda \Delta t K$$

(I is the identity matrix, and K is the tridiagonal matrix given in lecture).

This equation is an implicit solver that represents diffusion of radiation, and it gives the temperature of the area after a certain time step. The first thing I did to implement this into my code was define all the constants given in Table 1 in the lab manual. The full list of the constants I defined are as follows: solar flux, the heat capacity of seawater, the density of seawater, earth's radius, thermal diffusivity, the mixed-layer depth of the ocean, the Stefan-Boltzmann constant, the emissivity, and the albedos for ice/snow and for water/ground.

Next, I set up the grid that represents the earth with different latitude blocks. We were told to make the blocks 10° tall, so this means there will be 18 grid blocks (since latitude spans from 0° to 180°). I set these conditions to be true, and also placed the actual grid points in the middle of the blocks, rather than on the edges. Finally, I converted the latitude block sizes to be in radians rather than degrees, and the grid spacing to be in meters using the formula for arc length ($s = r\theta$). The last thing I set up before actually working with the matrices themselves is the time step and total time length. I set the total length of time to be 10,000 years in order to give the model enough time to reach a steady state.

Now that the basic initial conditions are defined and set, I could begin working with the matrices. I initialized K and B (B is a matrix that will be used in the following steps of Task 1) to be zeros, both the size of the number of latitude grid blocks. As stated before, K is the tridiagonal matrix we learned about in lecture. It represents the second derivative and how the radiation diffuses between latitude blocks. B is a matrix that represents the first derivative, which accounts for spherical correction by describing how temperature and cross-sectional area change with time on a sphere such as the earth.

I implemented a couple of for loops to set the values and conditions for these matrices. The first one implements the Neumann boundary conditions for the North and South Pole boundaries in matrix K, which marks where there is no heat flux. Otherwise, the loop goes through the rest of the cells and implements the radiation input or output relative to the cells around it. B will be used to account for the spherical shape of the earth; therefore, there are no boundary conditions (since the cross-sectional area at the poles is zero), and we can just set all the cells to expressions that take each of the surrounding cells into account.

Next, I found the spatial derivative of the cross-sectional area with respect to latitude. I did this by converting the latitudes to values in radians, computing the cross-sectional areas of the ocean mixed layer for each latitude and assigning them to an array (A_xz), and multiplying that array by matrix B to get the rate at which the surface area changes across latitudes.

Then, I set up an implicit time stepper using the equation written above. I defined the identity matrix using np.eye( ) to be put into the equation. Finally, I took the inverse of L so that it could be multiplied by other matrices to produce the target plots.

After all this set up, I was finally able to actually compute the temperatures of each latitude block on the earth with the additional factors and to plot them. For each curve that was plotted, I tested to make sure it looked accurate on its own before plotting them all together to match the figure in the lab manual. These test plots are included in the Analysis section of this report. The first line I plotted, the blue parabola, is just the raw temperatures that resulted from using the function temp_warm( ) (T_basic), which was pre-made and given to us in a separate file. For the rest of the three lines, I found the temperatures for all the corresponding latitudes using for loops, adding each additional condition as necessary. Next, I got the temperatures of each latitude with the diffusional properties taken into account by multiplying the inverse matrix L (L_inv) by T_basic, and I plotted this result in red. Then I added the spherical term to factor in earth's shape and its effects. The spherical term is just added to T_basic and that sum is multiplied by L_inv. The equation for the spherical term is given below.

$$f(T,\ A_{xz})j\ =\ \lambda\Delta t B_j B A_{xz}$$

The results of this process are plotted in yellow. Finally, the last condition to add is radiative forcing. This factor is given by the expression below.

$$\frac{\Delta t}{\rho C dz}(S(y)(1\ -\ \alpha)\ -\ \epsilon\sigma T^4)$$

Just like with the addition of the diffusion term, I added the radiative forcing term to T_basic to be multiplied by L_inv (keeping the diffusion term in that sum as well). This is the longest step in this whole plotting process because the calculations include basic temperature, diffusion, spherical properties, and radiative forcing all together. This final line is plotted in green. The resultant graph is shown in the Analysis section of this lab report.

Task 2

The goal for this task is to find the best values for diffusivity and emissivity that result in a warm-earth scenario (which is the curve produced by using temp_warm( ) and plotting its results). As instructed in the lab manual, I kept the albedo over the whole earth to be 0.3.

Next, I made variables called lam_test and emiss_test that I changed the values of as I tested which combination of values would match the temp_warm( ) curve best. I then almost completely repeated the steps from producing the green curve in Task 1: I calculated the matrix L using the new values being tested (tuned_L), and I also calculated the solar radiation using another given function, insolation( ). I put all of these new variable values into a similarly structured for loop as Task 1 where each of those condition terms are being calculated, then multiplied by tuned_L, and the results are put into an array to be graphed. The final values I

chose and the resultant plot of this technique overlaid with the control result (temp_warm( )'s curve) is shown in the Analysis section of this report.

Task 3

For this task, we varied the initial condition to investigate how the equilibrium solution changed. To do this, I wrote a function called dynamic_albedo( )that takes an initial temperature and an initial albedo and goes through the exact same for loop as in Task 2, but this time with the additional preceding step that determines the albedo the loop should use. The correct albedo value is chosen based on whether the surface is ice/snow or water/ground. That condition is checked in the code with a for loop that was given in the lab manual that tests each location against the threshold for ice (-10ºC or below) and assigns the cell the corresponding albedo value. These albedos are then used in the following for loop that calculates the temperatures at each latitude, just like the previous tasks. The function returns both an array of temperatures as well as the albedos for each latitude.

There are three different scenarios with different initial conditions to test: hot earth (everywhere is 60ºC), cold earth (everywhere is -60ºC), and a flash-frozen earth (use the warm earth temperatures from Tasks 1 and 2 with an initial albedo of 0.6). For all three of these cases, I initialized an array of temperatures by setting every latitude block to be the corresponding temperature of the test, chose the correct initial albedo value (for example, everywhere on the earth is frozen in the cold earth scenario, so we have to use the ice albedo as an initial albedo), and ran it through dynamic_albedo( ). I then plotted all three of the resultant curves on the same plot for comparison and analysis, which is included in the Analysis section of this report.

Task 4

For the final task of this lab, we want to investigate how solar forcing impacts the equilibrium solution. The solar forcing is varied by scaling S0 with a constant γ. Starting with a cold earth, the solar forcing begins at 0.4, steps up by 0.05 all the way up to 1.40, then steps back downward until it returns to 0.4.

To implement this, I began by defining arrays that describe the ascent and descent of the solar forcing scaler. To actually use those solar forcings, I wrote a function called diff_gammas( ) that does the exact same thing as dynamic_albedo( ), but this time, it varies the solar radiation input based on the value of γ rather than the albedos. The function's parameters are initial temperature, gamma value(s), and the desired number of steps. It returns the temperatures of the earth's latitudes after the specified time period in an array.

I also wrote a function called global_avg_T( ) that calculates the global average temperature. Given a temperature profile, this function will find the global average temperature by averaging the temperatures of each latitude weighted by their areas.

To run this simulation, I wrote two for loops – one for the ascending gammas and another for the descending gammas – that calculate the current temperatures with each step in gamma using diff_gammas( ) and add them to an array. The loops also calculate the global average temperature with each gamma step and add it to another array. Each step in gamma uses the

previous one as the starting point. I then took the resultant global average temperatures and gammas from this process and plotted them together, the upward steps and downward steps as separate curves so they can be compared. The plot for this task is included in the Analysis section of this report.

Analysis

Task 1

Figures 1-4 below show the resultant test plots from each additional condition added to the global temperature model. As labeled in the plots' titles, the first one shown is simply the initial condition, the next one is with basic diffusion added, then diffusion plus the spherical correction, and finally, the diffusion, spherical correction, and radiative forcing conditions together.

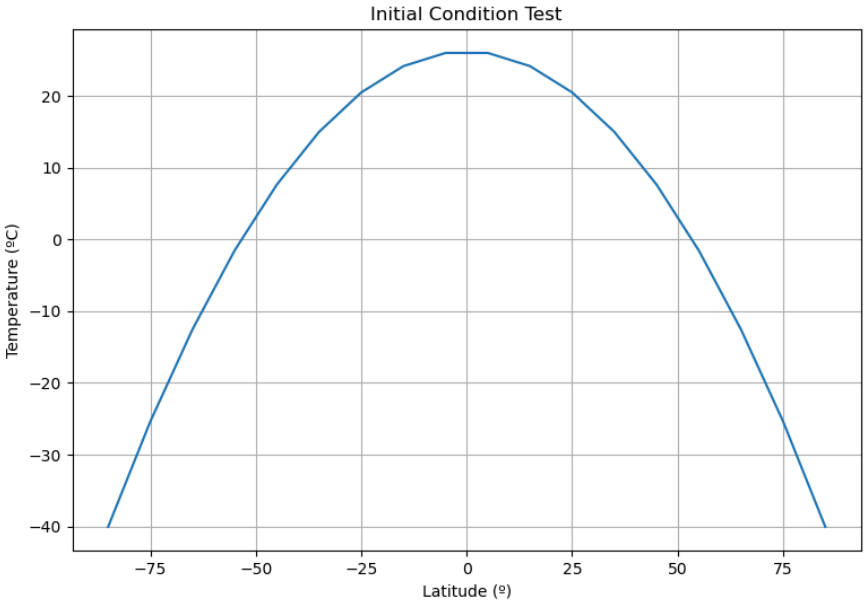


Figure 1

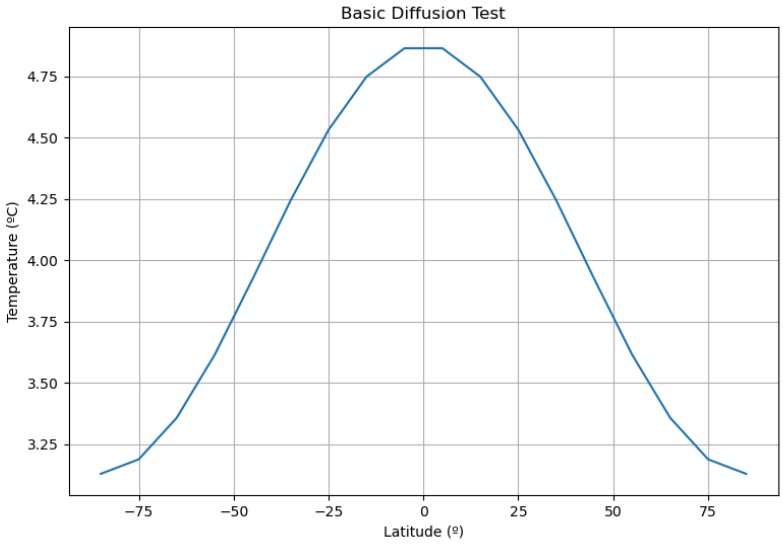


Figure 2

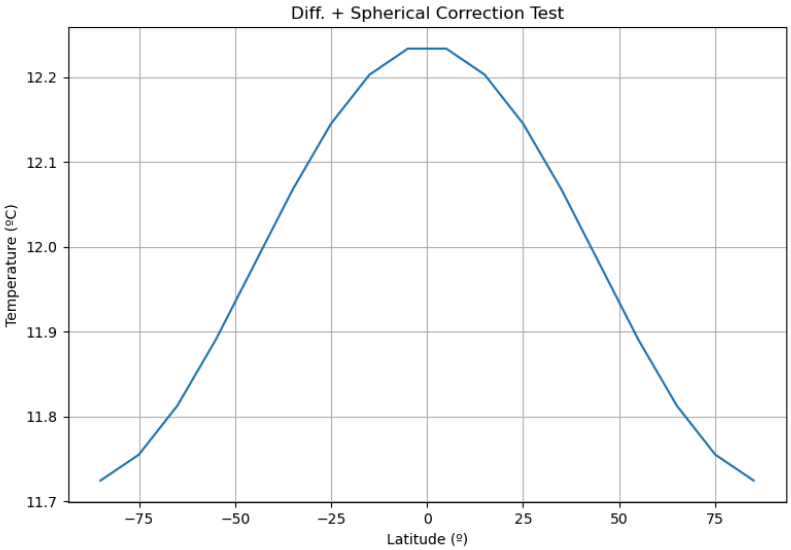


Figure 3

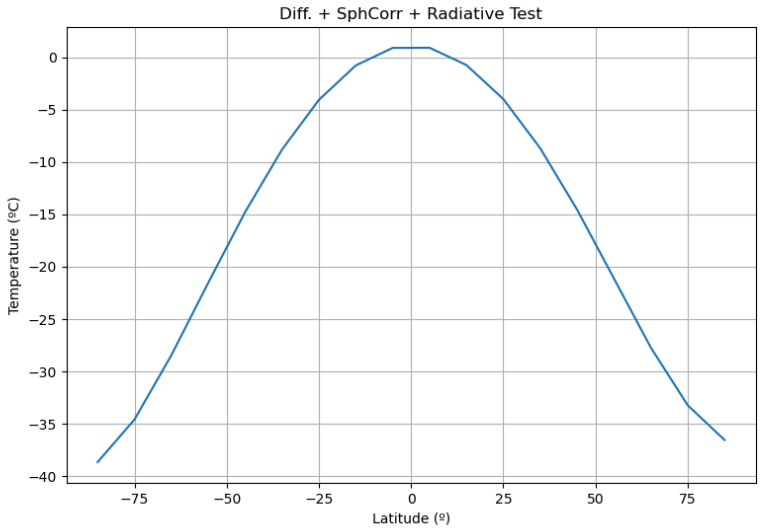


Figure 4

Comparing each result to the given figure in the manual, the values are reasonable for each individual case; so, I proceeded on to plot them all together. Figure 5 below is the plot produced to match the given figure in the lab report using the procedure outlined in the Methodology section of this report.

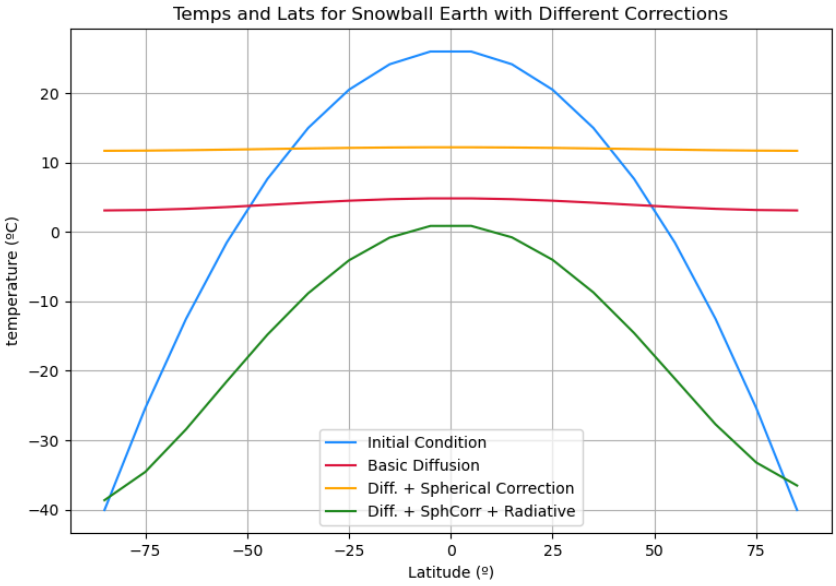


Figure 5

Task 2

After testing different combinations of diffusivity and emissivity, I determined a diffusivity of 55 and an emissivity of 0.7 to be the closest match for the control results from temp_warm( ). The graph that these values produced when put through the calculations is shown below in Figure 6 with the temp_warm( ) result also included on the graph.

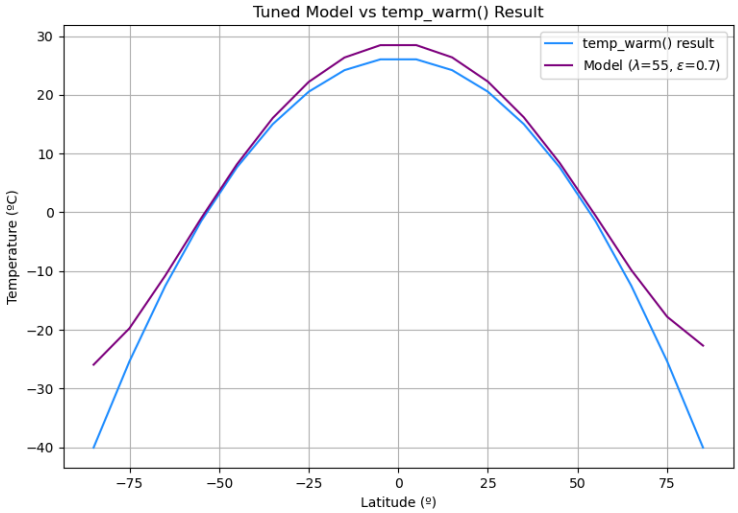


Figure 6

Task 3

Figure 7 shows the equilibrium solutions for each simulation with the different initial conditions: a cold earth (dark blue), a hot earth (light blue), and an earth that has been flash-freezed (medium blue).

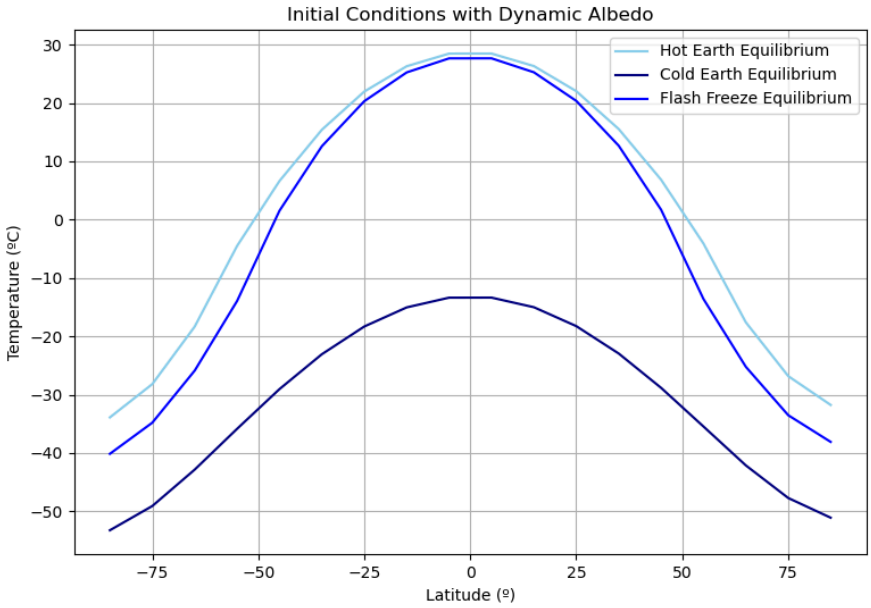


Figure 7

Based on this plot, it is evident that the flash freeze equilibrium is quite similar to the cold earth equilibrium, and the cold earth is much lower than the other two scenarios. Those two curves show that the temperature at the equator is around 27ºC, which is about 80ºF. This is a good temperature for the equator. In the midlatitudes (around 30ºN and S), the temperature is shown to be around 15ºC (59ºF), which is also a very reasonable temperature. The cold earth scenario shows an equator temperature of around -3ºC (26.6ºF) and a midlatitude temperature of around -20ºC (-4ºF), which are very low temperatures for earth equilibrium temperatures.

Both the hot earth and flash freeze earth converge to a similar stable equilibrium. This is because the solar insolation in combination with my chosen diffusivity and emissivity values were enough to overcome the sudden flash freeze of the earth, bringing it back to relatively normal conditions to match the hot earth scenario. The cold earth scenario is unstable. When the planet is partly frozen and near the threshold, the albedo is high enough that any incoming radiation is just reflected away, therefore lowering the temperature even more and creating a negative feedback loop.

Task 4

The snowball earth hypothesis represents a stable equilibrium. As shown in Figure 8, the solar multiplier increases, the global average temperature evens out and eventually stops changing. With such a high albedo, a big portion of the incoming solar radiation is reflected away, locking the planet in its glaciated state.

This plot reveals that there are two stable equilibria, and they occur on the glaciated branch and the warm branch. The unstable portion occurs around the tipping point, which determines the albedo of the planet and therefore controls the amount of solar radiation able to stay and warm the planet. Therefore, the stability of the model depends on the initial conditions.

The snowball earth hypothesis is valid based on my results. The plot shows that with a slightly lower solar multiplier, the planet can very well go into a glaciated state. Very far back in history, the sun was slightly dimmer. So, it can be reasonably concluded that it is possible for the earth to have been covered in glaciers at some point in history when the sun was dimmer than it is now.

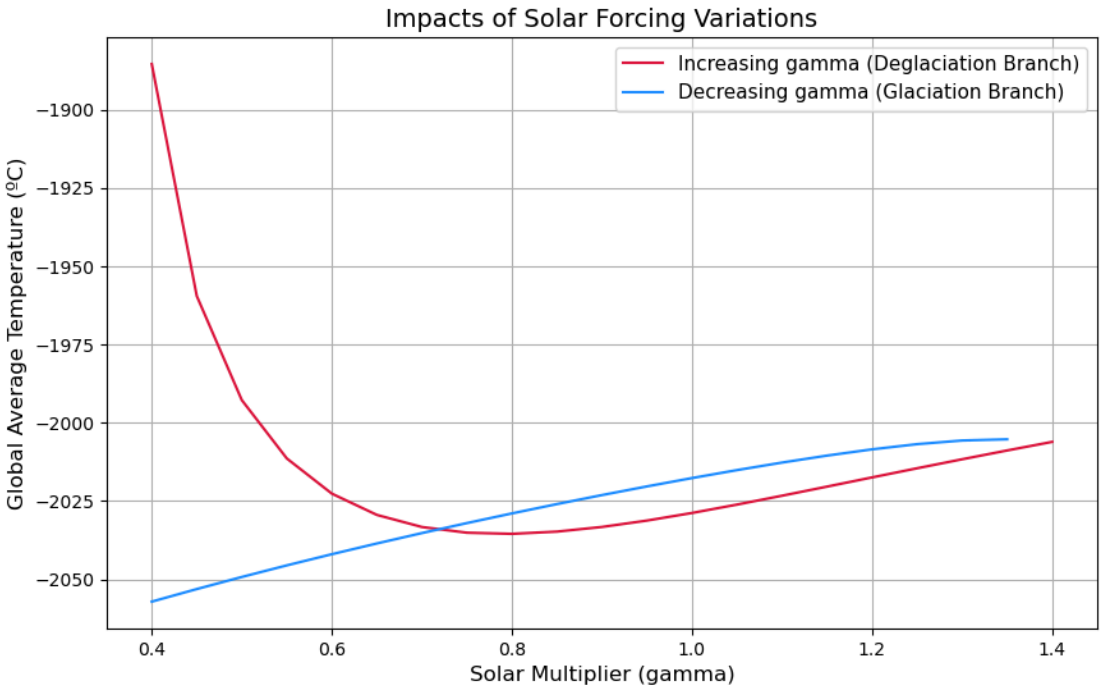

Impacts of Solar Forcing Variations
Increasing gamma (Deglaciation Branch)
Decreasing gamma (Glaciation Branch)
Global Average Temperature (ºC)
−1900
−1925
−1950
−1975
−2000
−2025
−2050
0.4
0.6
0.8
1.0
1.2
1.4
Solar Multiplier (gamma)

Figure 8